Exception Handling in Python – Complete Notes

1. What is Exception Handling?

Exception Handling is a process used to handle **runtime errors** so that the program does not crash unexpectedly.
It allows the program to continue running or show a proper message when an error occurs.

Example Runtime Errors:

- Division by zero
- Invalid input
- File not found
- Index out of range
- Type mismatch

2. Why Use Exception Handling?

Without exception handling:

```
num = 10 / 0
print("Hello")
```

Output:

```
ZeroDivisionError
```

Program stops immediately.

With exception handling:

```
try:
   num = 10 / 0
except:
   print("Error occurred")
```

Output:

Error occurred

Program continues safely.

3. Basic Syntax

```
try:
   # risky code
except:
   # runs if error occurs
```

Example:

```
try:
   print(10 / 0)
except:
   print("Cannot divide by zero")
```

4. Flow of Try-Except

1. Python executes code inside try
2. If no error → skip except
3. If error occurs → jump to except

5. Handling Specific Exceptions

Instead of using general except, use specific errors.

```
try:
   num = int("abc")
except ValueError:
   print("Invalid number")
```

Common Built-in Exceptions

Exception	Meaning
ZeroDivisionError	Dividing by zero
ValueError	Wrong value type
TypeError	Wrong data type
IndexError	Invalid list index
KeyError	Invalid dictionary key
FileNotFoundError	File missing
NameError	Variable not defined

6. Multiple Except Blocks

```
try:
    a = int(input("Enter number: "))
    print(10 / a)

except ValueError:
    print("Enter valid number")

except ZeroDivisionError:
    print("Cannot divide by zero")
```

7. Using else Block

else runs only when no exception occurs.

```
try:
    num = int(input("Enter number: "))
except ValueError:
    print("Invalid input")
else:
    print("You entered:", num)
```

8. Using finally Block

finally always runs whether error occurs or not.

```
try:
   f = open("data.txt")
except:
   print("File not found")
finally:
   print("Program ended")
```

Used for:

- Closing files
- Database cleanup
- Releasing resources

9. Full Structure

```
try:
   # risky code

except ErrorType:
   # handle error

else:
   # run if no error

finally:
   # always runs
```

10. Exception as Variable

Store error message in variable.

```
try:
   print(5 / 0)
except Exception as e:
   print("Error:", e)
```

Output:

```
Error: division by zero
```

11. Raising Exceptions Manually

Use raise.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

12. Custom Exceptions

Create your own exception class.

```
class InvalidAgeError(Exception):
    pass

age = -1

if age < 0:
    raise InvalidAgeError("Invalid age")
```

13. Nested Try Blocks

```
try:
    try:
        print(10 / 0)
    except ZeroDivisionError:
        print("Inner handled")
except:
    print("Outer handled")
```

14. Practical Example – ATM

```
try:
    balance = 5000
```

```
    amount = int(input("Enter withdraw amount: "))

    if amount > balance:
        raise Exception("Insufficient balance")

    print("Withdraw successful")

except Exception as e:
    print(e)
```

15. Best Practices

✅ Catch specific exceptions
✅ Use finally for cleanup
✅ Keep try block small
✅ Show meaningful messages
✅ Use custom exceptions in projects

❌ Avoid empty except:

```
except:
    pass
```

16. Real Life Uses

- Banking systems
- Login validation
- File handling
- APIs
- Database operations
- Payment systems

17. Short Notes for Exam

- try → risky code
- except → handles error

- else → runs if no error
- finally → always runs
- raise → create error manually
- Custom exception → user-defined error

18. Mini Practice Questions

Q1.

```
try:
   print(5/0)
except:
   print("Error")
```

Output? → Error

Q2.

Which block always runs?
Answer: finally

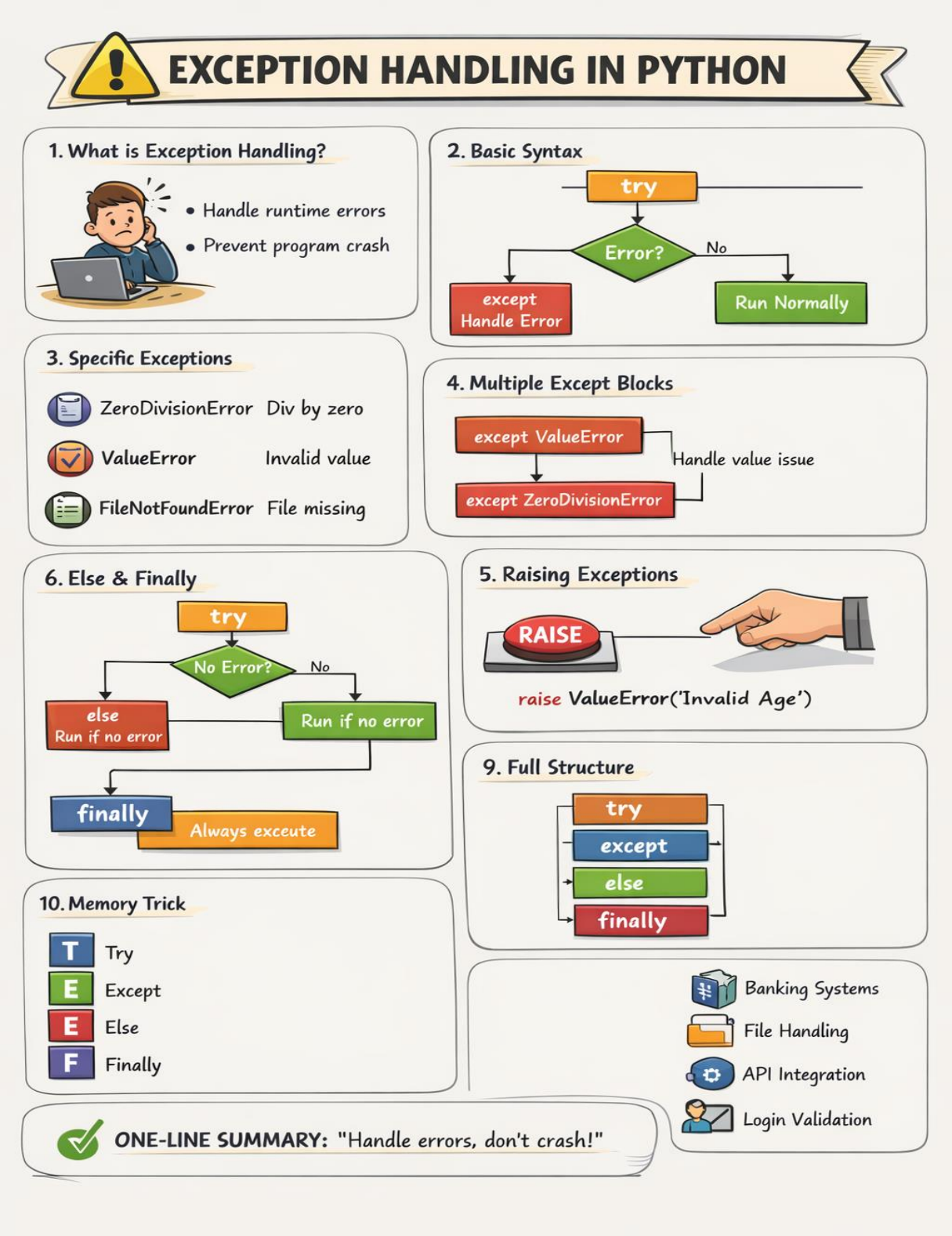
Q3.

Which keyword creates exception manually?
Answer: raise

19. Memory Trick

T E E F

- **T**ry
- **E**xcept
- **E**lse
- **F**inally

20. One-Line Summary

Exception Handling helps Python programs manage errors gracefully without crashing.